

Computer Architecture

A Complete Guide

Topics Covered

Von Neumann Model ▪ CPU Components ▪ ISA ▪ RISC vs CISC
Memory Hierarchy ▪ Cache ▪ Pipelining ▪ Hazards
Parallelism ▪ I/O & Buses ▪ Modern Techniques ▪ GPU
Architecture

Contents

1	Introduction to Computer Architecture	3
1.1	Three Levels of Abstraction	3
1.2	The CPU Performance Equation	3
2	Von Neumann Model	3
2.1	Architecture Diagram	4
2.2	The Von Neumann Bottleneck	4
2.3	Von Neumann vs Harvard Architecture	4
3	The CPU	4
3.1	Key Components	4
3.2	How the ALU Works	5
3.3	Register Categories	5
4	Fetch-Decode-Execute Cycle	5
4.1	The Four Stages	5
4.2	Worked Example: ADD R1, R2, R3	6
5	Instruction Set Architecture (ISA)	6
5.1	What the ISA Defines	6
5.2	Instruction Encoding (MIPS R-Type)	6
5.3	Addressing Modes	6
6	RISC vs CISC	6
6.1	Comparison	7
6.2	Modern x86 Internal Architecture	7
7	Memory Hierarchy	7
7.1	The Memory Pyramid	8
7.2	Memory Technologies	8
8	Cache Memory & Locality	8
8.1	Cache Hit vs Cache Miss	8
8.2	Cache Organization	9
8.3	Locality Principles	9
9	Pipelining	9
9.1	Classic 5-Stage RISC Pipeline	9
9.2	Pipeline Timing Diagram	9
10	Pipeline Hazards	10
10.1	Three Types of Hazards	10
10.2	Forwarding (Bypassing)	10
10.3	Branch Prediction	10
11	Parallelism	11
11.1	Flynn's Taxonomy	11

11.2 Instruction-Level Parallelism (ILP)	11
11.3 Amdahl's Law	11
12 I/O Systems & Buses	12
12.1 I/O Techniques	12
12.2 The Three Buses	12
12.3 Modern Interconnects	13
13 Modern CPU Techniques	13
13.1 Out-of-Order Execution	13
13.2 Speculative Execution	13
13.3 Typical Modern CPU Specifications	13
14 GPU Architecture	14
14.1 CPU vs GPU Design Philosophy	14
14.2 GPU Execution Hierarchy (NVIDIA)	14
14.3 GPU Memory Types	14
14.4 Why GPUs Dominate AI	15
Quick Reference Cheatsheet	16

1. Introduction to Computer Architecture

Computer architecture is the science of designing and organizing computer systems — from transistors and logic gates up to entire multi-processor systems. It defines how hardware components interact, how software runs on hardware, and how performance is maximized.

Architecture = ISA + Microarchitecture + System Design

1.1 Three Levels of Abstraction

ISA	Instruction Set Architecture — the contract between software and hardware. Defines what instructions exist and how they are encoded.
Microarchitecture	How the ISA is implemented in silicon — pipeline depth, cache sizes, and execution units.
System Architecture	How CPUs, memory, buses, and I/O devices are connected into a working system.

1.2 The CPU Performance Equation

Every architectural decision trades off three fundamental factors captured in the CPU performance equation:

$$\text{Execution Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Cycle}} \quad (1)$$

- **Instruction count** — reduced by better compilers, higher-level ISAs.
- **CPI (Cycles Per Instruction)** — reduced by pipelining, caches, OoO execution.
- **Clock period** — reduced by better silicon, deeper pipelines.

2. Von Neumann Model

Proposed by John von Neumann in 1945, this model describes a computer with a shared memory for both data and instructions, connected to a CPU via a single bus. It remains the foundation of nearly all modern computers.

2.1 Architecture Diagram

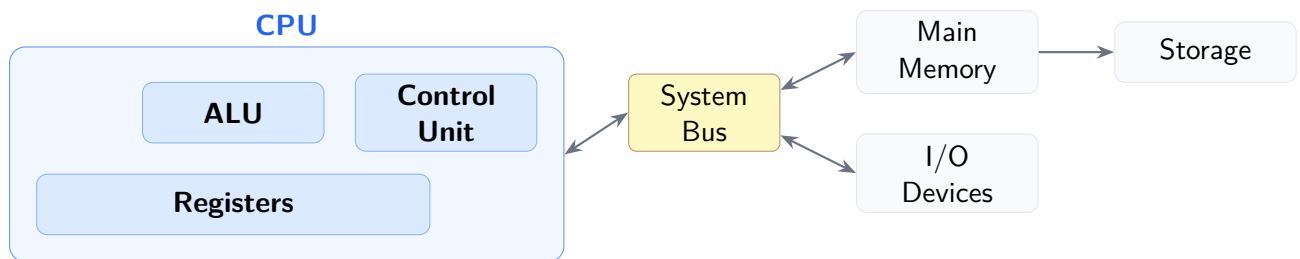


Figure 1: Von Neumann computer architecture

2.2 The Von Neumann Bottleneck

Because both instructions and data share the same bus, they cannot travel simultaneously. This creates a performance ceiling — the CPU frequently waits for memory. Modern architectures mitigate this with caches, wider buses, and Harvard-style split caches.

2.3 Von Neumann vs Harvard Architecture

Feature	Von Neumann	Harvard
Memory space	Single shared	Separate for code & data
Buses	One bus	Two independent buses
Flexibility	Programs can self-modify	Code is read-only
Bottleneck	Yes (shared bus)	No
Use cases	General-purpose CPUs	Microcontrollers, DSPs
Modern variant	Modified Harvard (split L1 cache)	Embedded cores (AVR, PIC)

3. The CPU

The Central Processing Unit (CPU) is the brain of the computer. It interprets and executes instructions, controls all other components, and performs arithmetic and logical operations.

3.1 Key Components

Component	Role	Key Detail
ALU	Arithmetic & Logic Unit — performs ADD, SUB, AND, OR, compare	Operates on register values; sets condition flags
Control Unit	Decodes instructions, coordinates data flow between components	Generates control signals for every unit
Registers	Ultra-fast on-chip storage	Typically 32 or 64 general-purpose registers

Component	Role	Key Detail
Program Counter (PC)	Holds address of next instruction	Auto-increments; jumps on branches
Instruction Register (IR)	Holds current instruction being executed	Fed by memory/cache after fetch
MAR / MDR	Memory Address / Data Registers — bus interface	MAR holds address; MDR holds data in transit
FPU	Floating Point Unit — decimal arithmetic	Separate from ALU; may be on- or off-chip
Clock	Synchronizes all CPU operations	Modern CPUs: 3–6 GHz; each tick = one cycle

3.2 How the ALU Works

The ALU receives two operands from registers, an operation code (opcode), and produces a result plus status flags:

```
ADD R1, R2, R3    ->    R1 = R2 + R3    (sets Zero, Carry,
                          Overflow, Negative flags)
```

3.3 Register Categories

Type	Description
General Purpose	Hold operands and intermediate results. Named R0–R31 (RISC) or EAX, EBX... (x86).
Special Purpose	PC, Stack Pointer (SP), Link Register (LR), Status Register (condition flags).
Floating Point	Separate bank for FP operations. 32 FP registers in MIPS; x87 stack in legacy x86.

4. Fetch-Decode-Execute Cycle

Every instruction a CPU runs goes through this fundamental sequence — billions of times per second.

4.1 The Four Stages

1. **Fetch** — PC → MAR → Memory → IR; PC = PC + 1
2. **Decode** — Control Unit reads IR; identifies opcode and operands
3. **Execute** — ALU operates / memory accessed / branch taken
4. **Write Back** — result written to register or memory

4.2 Worked Example: ADD R1, R2, R3

Stage	What Happens
Fetch	PC=100 → MAR=100 → memory[100] loaded into IR → PC=101
Decode	Control unit reads IR: opcode=ADD, dest=R1, src1=R2, src2=R3
Execute	ALU computes R2 + R3; sets flags (Zero, Carry, Overflow)
Write Back	ALU result stored into register R1

5. Instruction Set Architecture (ISA)

The ISA is the interface between software and hardware — it defines the vocabulary of commands a processor understands. Programs compiled for one ISA will not run on a different ISA without translation.

5.1 What the ISA Defines

- **Instructions** — arithmetic, logic, memory, control flow, I/O.
- **Data types** — integer sizes (8/16/32/64-bit), floating point (single/double), SIMD vectors.
- **Addressing modes** — how operands are located.
- **Registers** — count, roles, caller/callee-saved conventions.

5.2 Instruction Encoding (MIPS R-Type)



5.3 Addressing Modes

Mode	Syntax	Effective Address / Value
Immediate	ADD R1, #5	Operand is the literal value 5
Register	ADD R1, R2	Operand is the value in R2
Direct	LOAD R1, [1000]	Memory address 1000
Register Indirect	LOAD R1, [R2]	Address stored in R2
Base + Offset	LOAD R1, [R2+8]	R2 + 8 (used for arrays/structs)
PC-relative	BEQ label	PC + signed offset (branches)

6. RISC vs CISC

Two philosophies for ISA design that shaped the industry. The debate drove decades of innovation, and the answer turned out to be: both approaches converged.

6.1 Comparison

Feature	RISC	CISC
Examples	ARM, RISC-V, MIPS	x86, x86-64
Instruction size	Fixed (32-bit)	Variable (x86: 1–15 bytes)
Memory operands	Load/store only	Allowed in ALU ops
General registers	32+	8 (legacy x86)
Execution	Mostly single-cycle	Multi-cycle for complex ops
Pipeline	Simple to design	Complex
Who does the work	Compiler	Hardware
Code density	Larger binaries	More compact

Modern convergence: Modern x86 CPUs translate CISC instructions into micro-ops (RISC-like operations) internally. The ISA is CISC; the execution engine is RISC. This gives the best of both worlds.

6.2 Modern x86 Internal Architecture

Feature	Original RISC	Original CISC	Modern x86
Internal ops	Simple	Complex	RISC micro-ops
Visible registers	32+	8	16 + 100s internal
Instruction width	Fixed 32-bit	Variable	Variable (x86 ISA)
Pipeline	Simple 5-stage	Complex	20+ stage OoO

7. Memory Hierarchy

No single memory technology is simultaneously fast, large, and inexpensive. Computers therefore use a hierarchy: fast-small-expensive near the CPU, and slow-large-cheap further away.

7.1 The Memory Pyramid

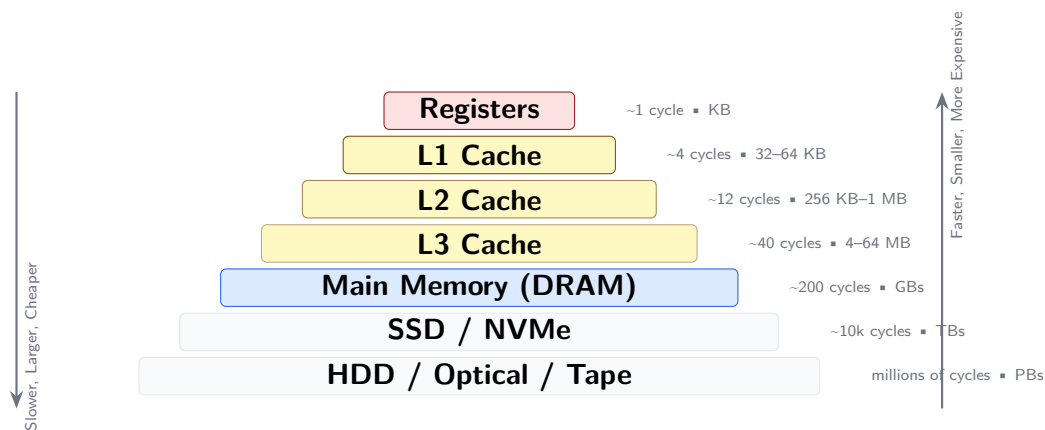


Figure 2: The memory hierarchy

7.2 Memory Technologies

Type	Technology	Volatile?	Primary Use
SRAM	Flip-flops (6 transistors/bit)	Yes	CPU caches — fast, expensive
DRAM	Capacitor + transistor per bit	Yes	Main memory — needs refresh every ms
DDR5	Double Data Rate SDRAM	Yes	Current RAM standard (up to 6400 MT/s)
NAND Flash	Floating gate transistors	No	SSDs, phones — limited write cycles
HDD	Magnetic platters + read head	No	Bulk storage — slow seek, high capacity

8. Cache Memory & Locality

Cache exploits the fact that programs tend to access the same memory locations repeatedly (*temporal locality*) and access nearby addresses sequentially (*spatial locality*).

8.1 Cache Hit vs Cache Miss

Cache Hit: Data found in cache. L1 $\approx$ 4 cycles, L2 $\approx$ 12 cycles, L3 $\approx$ 40 cycles. Modern programs hit L1 approximately 95% of the time.

Cache Miss: Data not in cache. Must fetch from DRAM (100–200 cycles). CPU stalls. This is the primary enemy of memory-bound performance.

8.2 Cache Organization

Concept	Explanation
Cache line	Smallest unit loaded from memory — typically 64 bytes. Even fetching 1 byte loads 64 (exploits spatial locality).
Direct-mapped	Each memory address maps to exactly one cache slot. Simple but causes conflict misses.
Set-associative	Each address maps to a set of N slots (N -way). Real CPUs use 4–16 way. Better hit rate.
Fully associative	Any address can go to any slot. Best hit rate; too expensive for large caches.
Write-through	Every write goes to cache <i>and</i> memory simultaneously. Consistent but slower.
Write-back	Writes go to cache only; memory updated when line is evicted. Faster; needs dirty bit.
Replacement policy	When cache is full, which line to evict? LRU (Least Recently Used) is standard.

8.3 Locality Principles

Temporal Locality A recently used address is likely to be used again soon (loop variables, counters, hot functions).

Spatial Locality Addresses near a recently used address are likely to be used soon (array elements, struct fields, sequential code).

9. Pipelining

Pipelining overlaps the execution of multiple instructions — like a factory assembly line. While one instruction executes, the next is being decoded, and another is being fetched.

9.1 Classic 5-Stage RISC Pipeline



Figure 3: 5-stage RISC pipeline stages

9.2 Pipeline Timing Diagram

	t1	t2	t3	t4	t5	t6	t7	t8
I1	IF	ID	EX	M	WB			
I2		IF	ID	EX	M	WB		
I3			IF	ID	EX	M	WB	
I4				IF	ID	EX	M	WB

With pipelining, 4 instructions complete in 8 cycles instead of 20 (4×5). At steady state, ideal throughput is **1 instruction per cycle**.

Deeper pipelines allow higher clock speeds but increase the penalty on branch mispredictions and introduce more hazard complexity.

10. Pipeline Hazards

Hazards are situations that prevent the next instruction from executing in the expected clock cycle, stalling or corrupting the pipeline.

10.1 Three Types of Hazards

Hazard	Cause	Solutions
Data (RAW)	Instruction needs result of a prior instruction not yet written back. Read-After-Write dependency.	Forwarding/bypassing; pipeline stalls (bubble insertion); compiler instruction reordering.
Control	Branch — CPU does not know which instruction to fetch next until branch resolves in EX stage.	Branch prediction; branch delay slots; speculative execution; pipeline flush on misprediction.
Structural	Two instructions need the same hardware resource simultaneously.	Duplicate resources (split caches); stall one instruction.

10.2 Forwarding (Bypassing)

Instead of waiting for WB to write a result before ID can read it, the result is forwarded directly from the EX output to the EX input of the dependent instruction:

```
ADD R1, R2, R3    <- produces R1
SUB R4, R1, R5    <- needs R1 immediately
    -> Forward EX output of ADD -> EX input of SUB (zero stalls)
```

10.3 Branch Prediction

The CPU guesses whether a branch will be taken before knowing for sure. Modern CPUs achieve approximately 99% accuracy.

Type	Description
Static prediction	Always predict not-taken, or use compiler hints. Simple, $\approx 60\%$ accurate.
2-bit saturating counter	Tracks recent branch history per branch. $\approx 90\%$ accurate.
Dynamic (TAGE, perceptron)	Uses global and local branch history patterns. $\approx 95\text{--}99\%$ accurate.

11. Parallelism

Modern performance gains come almost entirely from parallelism — from instruction-level to thread-level to data-level parallelism.

11.1 Flynn's Taxonomy

Class	Name	Description
SISD	Single Instruction, Single Data	Classic uniprocessor. One instruction, one data element.
SIMD	Single Instruction, Multiple Data	One instruction processes many elements (GPUs, AVX, NEON). Ideal for graphics and AI.
MISD	Multiple Instruction, Single Data	Rare; fault-tolerant systems verify the same data with multiple processors.
MIMD	Multiple Instruction, Multiple Data	Modern multi-core CPUs and clusters. Each processor runs independently.

11.2 Instruction-Level Parallelism (ILP)

Technique	How It Works
Superscalar	Multiple execution units run in parallel. Issue 2–6 instructions per cycle.
Out-of-Order (OoO)	CPU reorders instructions dynamically to avoid stalls, using a Reorder Buffer (ROB).
VLIW	Compiler packs multiple operations into one wide instruction word (Intel Itanium, DSPs).

11.3 Amdahl's Law

The speedup achievable with N processors when a fraction P of the workload can be parallelized:

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}} \quad (2)$$

Example: $P = 0.80$ (80% parallel), $N = 16$ cores:

$$S(16) = \frac{1}{0.20 + 0.80/16} = \frac{1}{0.25} = 4\times$$

The serial fraction $(1 - P)$ dominates at large N — doubling cores yields diminishing returns.

12. I/O Systems & Buses

I/O connects the CPU and memory to the outside world. It is the slowest part of every system and drives most architectural complexity.

12.1 I/O Techniques

Method	How It Works	Best For
Programmed I/O	CPU polls device status in a tight loop. Wastes CPU cycles.	Very simple embedded systems.
Interrupt-driven	Device signals CPU when ready via interrupt; CPU resumes other work until then.	Most I/O in general-purpose operating systems.
DMA	Direct Memory Access controller moves data without CPU involvement. CPU only sets up the transfer and receives a completion interrupt.	High-throughput: disk, GPU, network.

12.2 The Three Buses

Bus	Description
Address Bus	Carries memory address. Width determines addressable space: 32-bit = 4 GB; 64-bit = 16 EB.
Data Bus	Carries actual data. Width = word size (32 or 64 bits). Wider = more data per transfer.
Control Bus	Read/Write signals, interrupt lines, bus request/grant, clock synchronization.

12.3 Modern Interconnects

Standard	Peak Bandwidth	Use Case
PCIe 5.0 ×16	≈128 GB/s	GPU, NVMe SSD, high-speed NICs
DDR5-6400	≈51 GB/s	Main memory channel
USB 4 / Thunderbolt 4	40 Gbps	Peripherals, external storage
SATA III	≈600 MB/s	Legacy 2.5" SSDs, HDDs

13. Modern CPU Techniques

Today's processors use many advanced micro-architectural techniques to extract maximum performance from programs not explicitly written for parallelism.

13.1 Out-of-Order Execution

The CPU dynamically reorders instructions to avoid stalls. A **Reorder Buffer (ROB)** tracks in-flight instructions and commits them in-order to maintain correctness, while execution proceeds out-of-order.

OoO Engine Pipeline

Fetch → Decode → **Rename** → Dispatch → Issue Queue → **Execute (OoO)** → ROB Commit (in-order)

Register renaming eliminates false dependencies (WAR, WAW hazards) by mapping architectural registers to a large pool of physical registers.

13.2 Speculative Execution

The CPU executes instructions before knowing if they are needed (e.g., after a predicted branch). If prediction is correct: free performance. If wrong: results are discarded and the pipeline is flushed.

13.3 Typical Modern CPU Specifications

Feature	Typical Values (2024)
Pipeline depth	12–20 stages
Issue width (superscalar)	6–8 instructions/cycle
Reorder Buffer size	200–600 entries
L1 data cache	48–192 KB per core
Branch predictor accuracy	≈99%
L3 cache	4–64 MB (shared)
Prefetchers	Hardware prefetch for L1, L2, L3

14. GPU Architecture

GPUs are massively parallel processors — not faster at individual tasks, but capable of running thousands of simple tasks simultaneously. Originally designed for graphics, they are now essential for AI and scientific computing.

14.1 CPU vs GPU Design Philosophy

Property	CPU	GPU
Design goal	Minimize latency per thread	Maximize throughput across threads
Core count	8–64 powerful cores	Thousands of simpler cores
Cache size	Large (exploits locality)	Small per thread
Latency hiding	OoO execution, branch prediction	Switch to another warp
Memory bandwidth	50–100 GB/s (DRAM)	1–3.3 TB/s (HBM)

14.2 GPU Execution Hierarchy (NVIDIA)

Level	NVIDIA Term	Description
Streaming Multiprocessor	SM	Basic compute unit. Contains CUDA cores, shared memory, and schedulers. Runs many warps concurrently.
Warp	Warp	32 threads executing the same instruction in lockstep (SIMT model). GPU hides memory latency by switching warps.
Thread Block	CTA	Group of threads sharing fast on-chip shared memory. Scheduled to one SM.
Grid	Grid	All thread blocks in one kernel invocation. Can span all SMs on the device.

14.3 GPU Memory Types

Memory Type	Description
Global Memory	Large VRAM (8–80 GB), high bandwidth (up to 3.3 TB/s on H100), relatively high latency. Main working area.
Shared Memory	Per-SM programmer-managed scratchpad (48–228 KB). Very fast. Used for matrix tiling in GEMM operations.
Registers	Per-thread. GPUs have enormous register files (65,536 per SM in Ampere) to support many concurrent warps without spilling.
Constant / Texture	Read-only cached memory optimized for broadcast (all threads read the same address).

14.4 Why GPUs Dominate AI

Deep learning is dominated by matrix multiplications (GEMMs). These map perfectly to SIMD/tensor operations. Modern GPUs add dedicated **Tensor Cores** that perform 4×4 matrix multiply-accumulate in a single cycle, delivering up to 312 TFLOPS (FP16) on the NVIDIA H100 — roughly $60\times$ a high-end CPU.

Key insight: A GPU hides memory latency not with large caches (like a CPU) but by having enough warps in flight that when one warp stalls waiting for memory, another warp is ready to execute immediately. Latency hiding through massive thread-level parallelism.

Quick Reference Cheatsheet

CPU Performance Equation

$$T = IC \times CPI \times \tau$$

IC = instruction count, CPI = cycles per instruction, τ = clock period.

Amdahl's Law

$$S(N) = \frac{1}{(1 - P) + P/N}$$

P = parallel fraction, N = number of processors.

Pipeline Efficiency

Ideal $CPI = 1$. Actual $CPI = 1 + \text{stalls per instruction}$.

Cache Miss Penalty

$$AMAT = t_{L1} + m_{L1}(t_{L2} + m_{L2}(t_{L3} + m_{L3} \cdot t_{mem}))$$

AMAT = Average Memory Access Time.

Memory Capacity by Address Width

32-bit	4 GB
36-bit	64 GB
48-bit	256 TB
64-bit	16 EB

Common ISAs

x86-64	Intel/AMD PCs, servers
ARMv8/v9	Mobile, Apple Silicon
RISC-V	Open source, embedded
MIPS	Education, older routers

Pipeline Hazard Summary

RAW	Data hazard (most common)
WAR	Name hazard (renaming fixes)
WAW	Name hazard (renaming fixes)
Control	Branch hazard
Structural	Resource conflict
